

AnzenCore

Reporte de Auditoría de Seguridad de Aplicación Móvil

Resumen del Escaneo

Aplicación:	N/A
Paquete:	N/A
Versión:	N/A (Código: N/A)
Nombre de archivo:	uptodown-com.esaba.downloader.apk
Tamaño de archivo:	14.47 MB
Hash SHA-256:	863f05e8c42a2fbf870388bba81383c05469a387f1e6e92707198fe89cd38c5e
Fecha de análisis:	2026-06-18 01:17:55
Máxima Severidad:	Critico
Hallazgos totales:	6

Hallazgos de Seguridad

[CRITICO] Posible secreto hardcodeado

Tipo: hardcoded_secret | **Fuente:** META-INF/com/pierfrancescosoffritti/androidyoutubeplayer/core/verification.properties | **Referencias:** CWE-798 / M9

Explicación:

Se detectaron patrones compatibles con tokens, secrets o API keys embebidas en el código.

Lo que implica (Riesgo):

Los secretos embebidos en el código (API Keys, tokens, contraseñas) son de fácil acceso mediante descompilación básica. Un atacante puede extraerlos y utilizarlos para acceder a servicios en la nube de terceros, APIs privadas o bases de datos a nombre de la aplicación.

Recomendación:

Eliminar secretos del código fuente y moverlos a un backend seguro o a configuraciones protegidas.

```
**Solución en código sugerida (Android/Java):**
```java
// ■ Código vulnerable (Hardcoded):
// String apiKey = "1234567890abcdef1234567890abcdef";

// ■ Código seguro (Usar BuildConfig o Keystore):
// 1. En tu archivo local.properties añade: MY_API_KEY="123456..."
// 2. En build.gradle: buildConfigField "String", "API_KEY", properties.getProperty("MY_API_KEY")
// 3. En tu código Java/Kotlin:
String apiKey = BuildConfig.API_KEY;
```
```

Evidencia encontrada:

```
Código encontrado:
```java
#Tue Sep 24 22:28:45 PDT 2024
token=RLCF53***7GH2LM
```
```

[MEDIO] Librerías nativas detectadas

Tipo: native_code | **Referencias:** M7

Explicación:

El APK incluye código nativo, lo que puede ocultar lógica sensible o controles de seguridad.

Lo que implica (Riesgo):

El uso de código nativo (.so) dificulta la ingeniería inversa estándar de Java/Kotlin, pero también aumenta el riesgo de vulnerabilidades de desbordamiento de memoria (buffer overflow) y puede utilizarse para ocultar comportamiento malicioso que evade la detección estática básica.

Recomendación:

Analizar las librerías nativas con herramientas especializadas.

[BAJO] Dirección IP Hardcodeada detectada

Tipo: hardcoded_ip | **Fuente:** classes.dex | **Referencias:** CWE-200 / M9

Explicación:

Se detectó una dirección IP (0.1.2.3) dentro del código compilado.

Recomendación:

Evitar dejar direcciones IP internas o externas quemadas en el código fuente.
Extraer a variables de entorno de compilación o resolver mediante DNS.

Evidencia encontrada:

```
Archivo: classes.dex
Código encontrado:
```java
.
. . . ! " # $ % & ' () * + , - . / 0 1 2 3 4 5 6 7 8 9 : ; < = > ? @ A B C D E F G H I J K L M N O P Q
.R S T U V W X Y Z [\] ^ _ ` a b c d e f g h i j k l m n o p q r s t u v w x y z { | } ~ ■
...../■/■/■/■/■/■/■/ /
/
```
```

[BAJO] Dirección IP Hardcodeada detectada

Tipo: hardcoded_ip | Fuente: classes3.dex | Referencias: CWE-200 / M9

Explicación:

Se detectó una dirección IP (0.1.2.3) dentro del código compilado.

Recomendación:

Evitar dejar direcciones IP internas o externas quemadas en el código fuente.
Extraer a variables de entorno de compilación o resolver mediante DNS.

Evidencia encontrada:

```
Archivo: classes3.dex
Código encontrado:
```java
.
. . . ! " # $ % & ' () * + , - . / 0 1 2 3 4 5 6 7 8 9 : ; < = > ? @ A B C D E F G H I J K L M N O P Q R
.S.T.U.V.W.X.Y.Z.[.\.].^_` .a.b.c.d.e.f.g.h.i.j.k.l.m.n.o.p.q.r.s.t.u.v.w.x.y.z.{.|.}~.■.....
.....
...../■/■/■/■/■/■/ /
/
```
```

[BAJO] Dirección IP Hardcodeada detectada

Tipo: hardcoded_ip | Fuente: classes4.dex | Referencias: CWE-200 / M9

Explicación:

Se detectó una dirección IP (0.1.2.3) dentro del código compilado.

Recomendación:

Evitar dejar direcciones IP internas o externas quemadas en el código fuente.
Extraer a variables de entorno de compilación o resolver mediante DNS.

Evidencia encontrada:

```
Archivo: classes4.dex
Código encontrado:
```java
.
. . . ! " # $ % & ' () * + , - . / 0 1 2 3 4 5 6 7 8 9 : ; < = > ? @ A B C D E F G H I J K L M N O P Q
.R.S.T.U.V.W.X.Y.Z.[.\.].^_` .a.b.c.d.e.f.g.h.i.j.k.l.m.n.o.p.q.r.s.t.u.v.w.x.y.z.{.|.}~.■.....
.....
...../■/■/■/■/■/■/ /
/
```
```

[INFO] MultiDex detectado

Tipo: dex

Explicación:

Se detectaron 4 archivos DEX.

Lo que implica (Riesgo):

Los archivos DEX contienen el código compilado de la aplicación (Dalvik Executable). Su ausencia significa que la aplicación no contiene código ejecutable, lo cual indica que el archivo está corrupto o ha sido alterado.

Recomendación:

Revisar clases decompiladas durante el análisis profundo.

Evidencia encontrada:

classes.dex, classes2.dex, classes3.dex, classes4.dex

Artefactos de la Aplicación

| Tipo | Valor | Fuente |
|----------------|---|---|
| dex_count | 4 | None |
| file_count | 2062 | None |
| native_library | lib/arm64-v8a/libandroidx.graphics.path.so | lib/arm64-v8a/libandroidx.graphics.path.so |
| native_library | lib/arm64-v8a/libdatastore_shared_counter.so | lib/arm64-v8a/libdatastore_shared_counter.so |
| native_library | lib/arm64-v8a/libuptodown-native.so | lib/arm64-v8a/libuptodown-native.so |
| native_library | lib/armeabi-v7a/libandroidx.graphics.path.so | lib/armeabi-v7a/libandroidx.graphics.path.so |
| native_library | lib/armeabi-v7a/libdatastore_shared_counter.so | lib/armeabi-v7a/libdatastore_shared_counter.so |
| native_library | lib/armeabi-v7a/libuptodown-native.so | lib/armeabi-v7a/libuptodown-native.so |
| native_library | lib/x86/libandroidx.graphics.path.so | lib/x86/libandroidx.graphics.path.so |
| native_library | lib/x86/libdatastore_shared_counter.so | lib/x86/libdatastore_shared_counter.so |
| native_library | lib/x86/libuptodown-native.so | lib/x86/libuptodown-native.so |
| native_library | lib/x86_64/libandroidx.graphics.path.so | lib/x86_64/libandroidx.graphics.path.so |
| native_library | lib/x86_64/libdatastore_shared_counter.so | lib/x86_64/libdatastore_shared_counter.so |
| native_library | lib/x86_64/libuptodown-native.so | lib/x86_64/libuptodown-native.so |
| url | http://www.apache.org/licenses/ | META-INF/androidx/annotation/annotation/LICENSE.txt |
| url | http://schemas.android.com/apk/res/android | AndroidManifest.xml |
| url | http://schemas.android.com/apk/res-auto**http://schemas.android.com/apk/res/android | res/-7.xml |
| url | http://schemas.android.com/aapt**http://schemas.android.com/apk/res/android | res/16.xml |

Guía Técnica de Artefactos Extraídos

■ Cantidad de archivos DEX (dex_count)

Explicación: Indica el número de archivos Dalvik Executable (DEX) compilados dentro de la aplicación. Los archivos DEX contienen el código fuente de Java/Kotlin compilado en bytecode ejecutable por la máquina virtual de Android.

Recomendación: Si el número de archivos DEX es alto (MultiDex), indica una base de código grande. Se recomienda habilitar la minificación y obfuscación (ProGuard/R8) en el archivo build.gradle para eliminar código no utilizado y reducir el tamaño total del APK.

■ Número de archivos totales (file_count)

Explicación: El recuento total de archivos (código compilado, recursos, imágenes, layouts XML, manifiestos) contenidos en el paquete APK comprimido.

Recomendación: Auditar periódicamente los recursos del proyecto. Elimine imágenes, layouts u otros archivos no utilizados (usando herramientas como Android Lint o habilitando 'shrinkResources' en Gradle) para optimizar el peso del APK.

■ Biblioteca nativa (.so) (native_library)

Explicación: Archivos binarios de biblioteca compartida compilados en C/C++ usando el NDK para arquitecturas específicas de hardware (como ARM64 o x86). Se utilizan habitualmente para procesamiento de alta velocidad o integración de código binario de bajo nivel.

Recomendación: Dado que las librerías nativas pueden ocultar código malicioso y son susceptibles a fallos de desbordamiento de memoria, se aconseja desensamblarlas (con Ghidra o IDA Pro) y auditar de forma estática su comportamiento y llamadas a APIs nativas de red.

■ URL encontrada (url)

Explicación: Direcciones web o endpoints descubiertos dentro de las cadenas de texto del código o los recursos del APK. Suelen corresponder a APIs de backend de la aplicación, recursos externos, licencias de librerías o dominios de terceros.

Recomendación: Verificar que todas las URLs detectadas pertenezcan a servidores o servicios legítimos controlados. Asegúrese de que todos los endpoints utilicen HTTPS en producción y bloquee conexiones a hosts no reconocidos.
